

CELEBAL SUMMER INTERNSHIP 2026

WEEK 3 ASSIGNMENT

Advanced SQL using Subqueries, CTEs and Window Functions

Submitted By: Harshit Goel Technology: MySQL Dataset: Superstore Dataset

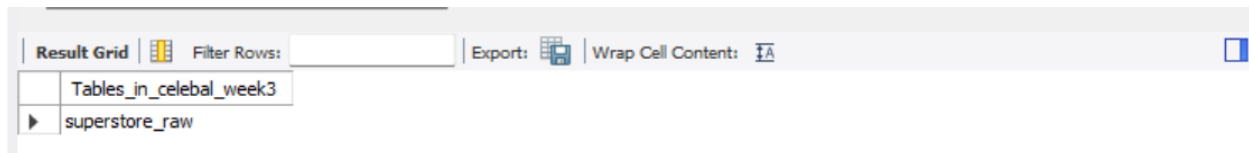
1. Dataset Import & Verification

1.1 Show Tables

SQL Query



Output



1.2 Describe Raw Table (superstore_raw)

SQL Query



Output

Result Grid						
Filter Rows:						
Export:						
Wrap Cell Content:						
Field	Type	Null	Key	Default	Extra	
Row ID	int	YES		NULL		
Order ID	text	YES		NULL		
Order Date	text	YES		NULL		
Ship Date	text	YES		NULL		
Ship Mode	text	YES		NULL		
Customer ID	text	YES		NULL		
Customer Name	text	YES		NULL		
Segment	text	YES		NULL		
Country	text	YES		NULL		
City	text	YES		NULL		
State	text	YES		NULL		
Postal Code	int	YES		NULL		
Region	text	YES		NULL		
Product ID	text	YES		NULL		
Category	text	YES		NULL		
Sub-Category	text	YES		NULL		
Product Name	text	YES		NULL		
Sales	double	YES		NULL		
Quantity	int	YES		NULL		
Discount	double	YES		NULL		
Profit	double	YES		NULL		

1.3 Preview Dataset

SQL Query

SQL File 3	
Limit to 1000 rows	
1 • SELECT * FROM superstore_raw LIMIT 5;	

Output

Result Grid													
Filter Rows:													
Export:													
Wrap Cell Content:													
Fetch rows:													
Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	
3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	

1.4 Total Records in Dataset

SQL Query

SQL File 3	
Limit to 1000 rows	
1 • SELECT COUNT(*) AS total_records FROM superstore_raw;	

Output

Result Grid	
Filter Rows:	
Export:	
Wrap Cell Content:	
total_records	
9694	

2. Database Normalization

2.1 Create Customers Table

SQL Query

```
1 CREATE TABLE customers (  
2     customer_id VARCHAR(20) PRIMARY KEY,  
3     customer_name VARCHAR(100),  
4     segment VARCHAR(50),  
5     country VARCHAR(50),  
6     city VARCHAR(50),  
7     state VARCHAR(50),  
8     postal_code INT,  
9     region VARCHAR(50)  
10 );
```

Output

27 18:34:35 CREATE TABLE customers (customer_id VARCHAR(20) PRIMARY KEY, customer_name VAR... 0 row(s) affected 0.047 sec

2.2 Create Orders Table

SQL Query

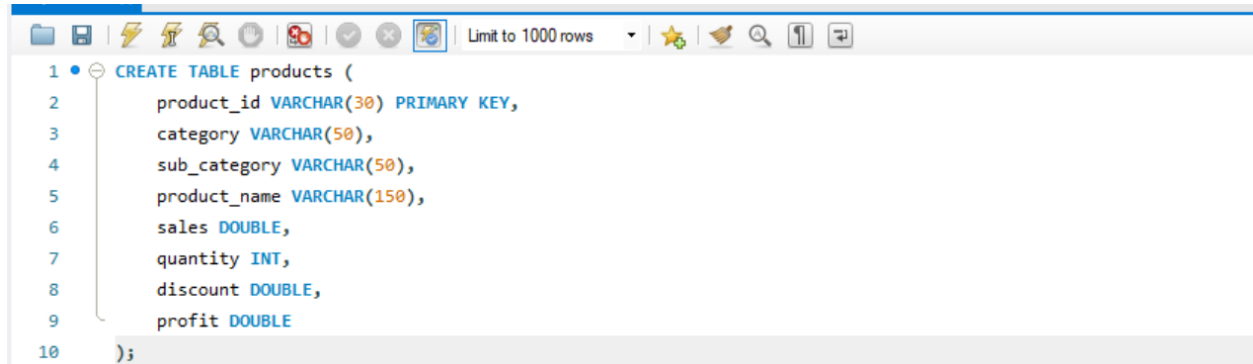
```
1 CREATE TABLE orders (  
2     order_id VARCHAR(20) PRIMARY KEY,  
3     order_date VARCHAR(20),  
4     ship_date VARCHAR(20),  
5     ship_mode VARCHAR(50),  
6     customer_id VARCHAR(20),  
7     FOREIGN KEY (customer_id)  
8         REFERENCES customers(customer_id)  
9 );
```

Output

28 18:36:00 CREATE TABLE orders (order_id VARCHAR(20) PRIMARY KEY, order_date VARCHAR(20), ... 0 row(s) affected 0.047 sec

2.3 Create Products Table

SQL Query



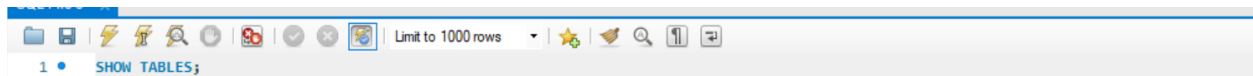
```
1 • CREATE TABLE products (  
2     product_id VARCHAR(30) PRIMARY KEY,  
3     category VARCHAR(50),  
4     sub_category VARCHAR(50),  
5     product_name VARCHAR(150),  
6     sales DOUBLE,  
7     quantity INT,  
8     discount DOUBLE,  
9     profit DOUBLE  
10 );
```

Output

✓ 29 18:37:10 CREATE TABLE products (product_id VARCHAR(30) PRIMARY KEY, category VARCHAR(50), ... 0 row(s) affected 0.062 sec

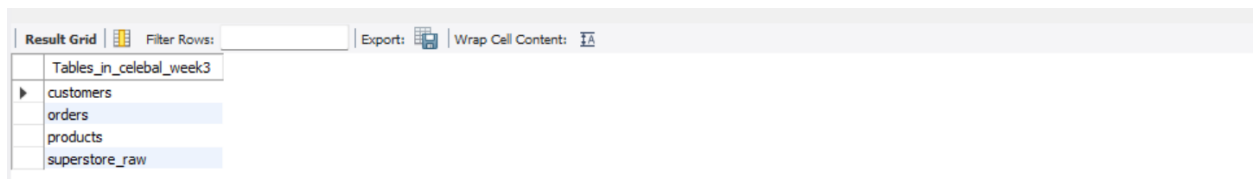
2.4 Verify Created Tables

SQL Query



```
1 • SHOW TABLES;
```

Output



Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Tables_in_celebal_week3			
▶	customers		
	orders		
	products		
	superstore_raw		

2.5 Describe Customers Table

SQL Query

```
1 DESCRIBE customers;
```

Output

Field	Type	Null	Key	Default	Extra
customer_id	varchar(20)	NO	PRI	NULL	
customer_name	varchar(100)	YES		NULL	
segment	varchar(50)	YES		NULL	
country	varchar(50)	YES		NULL	
city	varchar(50)	YES		NULL	
state	varchar(50)	YES		NULL	
postal_code	int	YES		NULL	
region	varchar(50)	YES		NULL	

2.6 Describe Orders Table

SQL Query

```
1 DESCRIBE orders;
```

Output

Field	Type	Null	Key	Default	Extra
order_id	varchar(20)	NO	PRI	NULL	
order_date	varchar(20)	YES		NULL	
ship_date	varchar(20)	YES		NULL	
ship_mode	varchar(50)	YES		NULL	
customer_id	varchar(20)	YES	MUL	NULL	

2.7 Describe Products Table

SQL Query

```
1 DESCRIBE products;
```

Output

Field	Type	Null	Key	Default	Extra
product_id	varchar(30)	NO	PRI	NULL	
category	varchar(50)	YES		NULL	
sub_category	varchar(50)	YES		NULL	
product_name	varchar(150)	YES		NULL	
sales	double	YES		NULL	
quantity	int	YES		NULL	
discount	double	YES		NULL	
profit	double	YES		NULL	

2.8 Insert Customer Records

SQL Query

```
1  INSERT INTO customers (
2      customer_id,
3      customer_name,
4      segment,
5      country,
6      city,
7      state,
8      postal_code,
9      region
10 )
11 SELECT
12     `Customer ID`,
13     MAX(`Customer Name`),
14     MAX(Segment),
15     MAX(Country),
16     MAX(City),
17     MAX(State),
18     MAX(`Postal Code`),
19     MAX(Region)
20 FROM superstore_raw
21 GROUP BY `Customer ID`;
```

Output

34 18.54.52 INSERT INTO customers (customer_id, customer_name, segment, country, city, state,... 793 row(s) affected Records: 793 Duplicates: 0 Warnings: 0 0.156 sec

2.9 Verify Customers Table

SQL Query

```
1 • SELECT COUNT(*) AS total_customers FROM customers;
```

Output

Result Grid	
total_customers	793

2.10 Preview Customers Table

SQL Query

```
1 • SELECT * FROM customers LIMIT 5;
```

Output

	customer_id	customer_name	segment	country	city	state	postal_code	region
▶	AA-10315	Alex Avila	Consumer	United States	San Francisco	Texas	94122	West
	AA-10375	Allen Arnold	Consumer	United States	Salem	Tennessee	97301	West
	AA-10480	Andrew Allen	Consumer	United States	Springfield	North Carolina	65807	South
	AA-10645	Anna Andreadi	Consumer	United States	San Francisco	Pennsylvania	94109	West
	AB-10015	Aaron Bergman	Consumer	United States	Seattle	Washington	98103	West
*	HOLE	HOLE	HOLE	HOLE	HOLE	HOLE	HOLE	HOLE

2.11 Insert Order Records

SQL Query

```
1 INSERT INTO orders (
2     order_id,
3     order_date,
4     ship_date,
5     ship_mode,
6     customer_id
7 )
8 SELECT
9     `Order ID`,
10    MAX(`Order Date`),
11    MAX(`Ship Date`),
12    MAX(`Ship Mode`),
13    MAX(`Customer ID`)
14 FROM superstore_raw
15 GROUP BY `Order ID`;
```

Output

37 19:00:44 INSERT INTO orders (order_id, order_date, ship_date, ship_mode, customer_id) SELEC... 4931 row(s) affected Records: 4931 Duplicates: 0 Warnings: 0 0.437 sec

2.12 Verify Orders Table

SQL Query

```
1 SELECT COUNT(*) AS total_orders FROM orders;
```

Output

	total_orders
▶	4931

2.13 Preview Orders Table

SQL Query

```
1 SELECT * FROM orders LIMIT 5;  
2
```

Output

order_id	order_date	ship_date	ship_mode	customer_id
CA-2014-100006	9/7/2014	9/13/2014	Standard Class	DK-13375
CA-2014-100090	7/8/2014	7/12/2014	Standard Class	EB-13705
CA-2014-100293	3/14/2014	3/18/2014	Standard Class	NF-18475
CA-2014-100363	4/8/2014	4/15/2014	Standard Class	JM-15655
CA-2014-100391	5/25/2014	5/29/2014	Standard Class	BW-11065
NULL	NULL	NULL	NULL	NULL

2.14 Insert Product Records

SQL Query

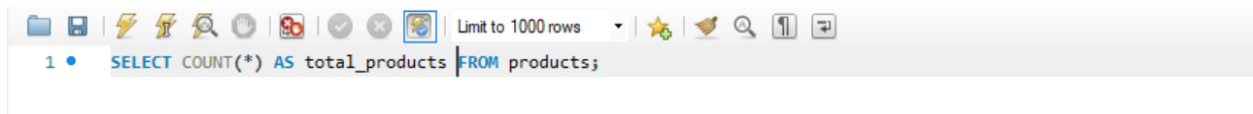
```
1 INSERT INTO products (  
2     product_id,  
3     category,  
4     sub_category,  
5     product_name,  
6     sales,  
7     quantity,  
8     discount,  
9     profit  
10 )  
11 SELECT  
12     `Product ID`,  
13     MAX(Category),  
14     MAX(`Sub-Category`),  
15     MAX(`Product Name`),  
16     MAX(Sales),  
17     MAX(Quantity),  
18     MAX(Discount),  
19     MAX(Profit)  
20 FROM superstore_raw  
21 GROUP BY `Product ID`;
```

Output

40 19:05:13 INSERT INTO products (product_id, category, sub_category, product_name, sales, q... 1812 row(s) affected Records: 1812 Duplicates: 0 Warnings: 0 0.343 sec

2.15 Verify Products Table

SQL Query

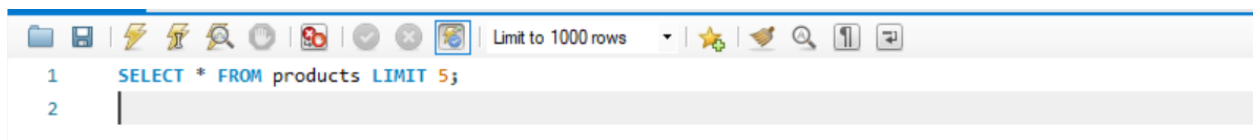


Output

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
total_products			
1812			

2.16 Preview Products Table

SQL Query



Output

[illegible]

3. Required SQL Queries

Q1. Find all orders where sales are greater than the average sales (Subquery)

SQL Query

```
1 • SELECT * FROM superstore_raw WHERE Sales >
2 (
3     SELECT AVG(Sales)
4     FROM superstore_raw
5 )
```

Output

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311
8	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032
11	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032
12	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032
14	CA-2016-161389	12/5/2016	12/10/2016	Standard Class	IM-15070	Irene Maddox	Consumer	United States	Seattle	Washington	98103
17	CA-2014-105893	11/11/2014	11/18/2014	Standard Class	PK-19075	Pete Kriz	Consumer	United States	Madison	Wisconsin	53711
25	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	Emily Burns	Consumer	United States	Orem	Utah	84057
28	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-215; EB-13870	Blumstein	Consumer	United States	Philadelphia	Pennsylvania	19140
36	CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	Gene Hale	Corporate	United States	Richardson	Texas	75080
39	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	Steve Nguyen	Home Of...	United States	Houston	Texas	77041
41	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	Steve Nguyen	Home Of...	United States	Houston	Texas	77041
55	CA-2016-105816	12/11/2016	12/17/2016	Standard Class	JM-15265	Janet Molinari	Corporate	United States	New York City	New York	10024
39	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	Steve Nguyen	Home Of...	United States	Houston	Texas	77041
41	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	Steve Nguyen	Home Of...	United States	Houston	Texas	77041
55	CA-2016-105816	12/11/2016	12/17/2016	Standard Class	JM-15265	Janet Molinari	Corporate	United States	New York City	New York	10024
58	CA-2016-111682	6/17/2016	6/18/2016	First Class	TB-21055	Ted Butterfield	Consumer	United States	Troy	New York	12180
68	CA-2014-106376	12/5/2014	12/10/2014	Standard Class	BS-11590	Brendan Sweed	Corporate	United States	Gilbert	Arizona	85234
73	US-2015-134026	4/26/2015	5/2/2015	Standard Class	JE-15745	Joel Eaton	Consumer	United States	Memphis	Tennessee	38109
86	CA-2017-140088	5/28/2017	5/30/2017	Second Class	PO-18865	Patrick O'Donnell	Consumer	United States	Columbia	South Carolina	29203
104	US-2015-156867	11/13/2015	11/17/2015	Standard Class	LC-16870	Lena Cacioppo	Consumer	United States	Aurora	Colorado	80013
110	CA-2015-129476	10/15/2015	10/20/2015	Standard Class	PA-19060	Pete Armstrong	Home Of...	United States	Orland Park	Illinois	60462
118	CA-2015-110457	3/2/2015	3/6/2015	Standard Class	DK-13090	Dave Kipp	Consumer	United States	Seattle	Washington	98103
125	US-2014-152030	12/26/2014	12/28/2014	Second Class	AD-10180	Alan Dominguez	Home Of...	United States	Houston	Texas	77041
126	US-2014-134614	9/20/2014	9/25/2014	Standard Class	PF-19165	Philip Fox	Consumer	United States	Bloomington	Illinois	61701
128	US-2017-107272	11/5/2017	11/12/2017	Standard Class	TS-21610	Troy Staebel	Consumer	United States	Phoenix	Arizona	85023
130	US-2016-125969	11/6/2016	11/10/2016	Second Class	LS-16975	Lindsay Shagiari	Home Of...	United States	Los Angeles	California	90004

Q2. Find the highest sales order for each customer (Subquery)

SQL Query

```
1 • SELECT
2     s.`Customer ID`,
3     s.`Customer Name`,
4     s.`Order ID`,
5     s.Sales
6 FROM superstore_raw s
7 JOIN (
8     SELECT
9         `Customer ID`,
10        MAX(Sales) AS max_sales
11    FROM superstore_raw
12    GROUP BY `Customer ID`
13 ) m
14 ON s.`Customer ID` = m.`Customer ID`
15 AND s.Sales = m.max_sales;
```

Output

	Customer ID	Customer Name	Order ID	Sales
▶	CG-12520	Claire Gute	CA-2016-152156	731.94
	SO-20335	Sean O'Donnell	US-2015-108966	957.5775
	BH-11710	Brosina Hoffman	CA-2014-115812	1706.184
	EB-13870	Emily Burns	CA-2015-106320	1044.63
	TB-21520	Tracy Blumstein	US-2015-150630	3083.43
	GH-14485	Gene Hale	CA-2016-117590	1097.544
	SN-20710	Steve Nguyen	CA-2015-117415	532.3992
	JM-15265	Janet Molinari	CA-2016-105816	1029.95
	TB-21055	Ted Butterfield	CA-2016-111682	319.41
	BS-11590	Brendan Sweed	CA-2014-106376	1113.024
	PA-19060	Pete Armstrong	CA-2015-129476	339.96
	HA-14920	Helen Andreada	CA-2015-110744	671.93
	JE-16165	Justin Ellison	CA-2016-114489	1951.84
	BM-11140	Becky Martin	CA-2014-139892	8159.952

	Customer ID	Customer Name	Order ID	Sales
	AH-10195	Alan Haines	CA-2016-112697	961.48
	CR-12625	Corey Roper	CA-2014-111003	289.2
	NM-18445	Nathan Mautz	CA-2014-164973	3991.98
	DB-13615	Doug Bickford	CA-2015-137946	959.984
	AC-10420	Alyssa Crouse	CA-2014-129924	698.352
	CC-12550	Clay Cheatham	CA-2017-169901	47.976
	AB-10060	Adam Bellavance	CA-2016-129714	4355.168
	MK-17905	Michael Kennedy	CA-2016-155516	1043.92
	AG-10900	Arthur Gainer	US-2014-119137	1023.936
	AR-10405	Allen Rosenblatt	US-2017-134481	1488.424
	DO-13435	Denny Ordway	US-2015-168935	375.4575
	JG-15115	Jack Garza	US-2014-135972	1799.97
	AG-10675	Anna Gayman	CA-2016-108987	2396.2656
	KH-16510	Keith Herrera	CA-2017-117457	1336.829

Q3. Calculate total sales for each customer (CTE)

SQL Query

```
SQL File 3
Limit to 1000 rows

1 WITH customer_sales AS (
2     SELECT
3         `Customer ID`,
4         `Customer Name`,
5         SUM(Sales) AS total_sales
6     FROM superstore_raw
7     GROUP BY
8         `Customer ID`,
9         `Customer Name`
10 )
11 SELECT *
12 FROM customer_sales
13 ORDER BY total_sales DESC;
```

Output

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
	Customer ID	Customer Name	total_sales
▶	SM-20320	Sean Miller	25043.05
	TC-20980	Tamara Chand	19017.847999999998
	RB-19360	Raymond Buch	15117.339
	TA-21385	Tom Ashbrook	14595.62
	AB-10105	Adrian Barton	14355.610999999997
	SC-20095	Sanjit Chand	14142.333999999999
	KL-16645	Ken Lonsdale	14071.917
	HL-15040	Hunter Lopez	12873.297999999999
	SE-20110	Sanjit Engle	12209.438000000002
	CC-12370	Christopher Conant	12129.072
	TS-21370	Todd Sumrall	11885.871
	GT-14710	Greg Tran	11820.119999999997
	BM-11140	Becky Martin	11609.900000000001
	SV-20365	Seth Vernon	11420.645999999997

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
	Customer ID	Customer Name	total_sales
	EH-13765	Edward Hooks	10005.980000000001
	JL-15835	John Lee	9403.003
	GT-14635	Grant Thornton	9351.211999999998
	HW-14935	Helen Wasserman	9300.253999999997
	TB-21400	Tom Boeckenhauer	9012.741999999998
	PF-19120	Peter Fuller	8980.496
	CM-12385	Christopher Martinez	8954.02
	JD-16150	Justin Deggeller	8828.030499999999
	JE-15715	Joe Elijah	8672.163
	LA-16780	Laura Armstrong	8631.521999999999
	DR-12940	Daniel Raglin	8350.868
	NF-18385	Natalie Fritzler	8322.826000000001
	KD-16270	Karen Daniels	8282.358
	NC-18535	Nick Crebassa	8241.739

Q4. Find customers whose total sales are above average (CTE + Subquery)

SQL Query

```
SQL File 3
Limit to 1000 rows

1 WITH customer_sales AS (
2     SELECT
3         `Customer ID`,
4         `Customer Name`,
5         ROUND(SUM(Sales), 2) AS total_sales
6     FROM superstore_raw
7     GROUP BY
8         `Customer ID`,
9         `Customer Name`
10 )
11 SELECT *
12 FROM customer_sales
13 WHERE total_sales >
14 (
15     SELECT AVG(total_sales)
16     FROM customer_sales
17 )
18 ORDER BY total_sales DESC;
```

Output

Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
	Customer ID	Customer Name	total_sales
▶	SM-20320	Sean Miller	25043.05
	TC-20980	Tamara Chand	19017.85
	RB-19360	Raymond Buch	15117.34
	TA-21385	Tom Ashbrook	14595.62
	AB-10105	Adrian Barton	14355.61
	SC-20095	Sanjit Chand	14142.33
	KL-16645	Ken Lonsdale	14071.92
	HL-15040	Hunter Lopez	12873.3
	SE-20110	Sanjit Engle	12209.44
	CC-12370	Christopher Conant	12129.07
	TS-21370	Todd Sumrall	11885.87
	GT-14710	Greg Tran	11820.12
	BM-11140	Becky Martin	11609.9
	SV-20365	Seth Vernon	11420.65

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Customer ID	Customer Name	total_sales	
JL-15835	John Lee	9403	
GT-14635	Grant Thornton	9351.21	
HW-14935	Helen Wasserman	9300.25	
TB-21400	Tom Boeckenhauer	9012.74	
PF-19120	Peter Fuller	8980.5	
CM-12385	Christopher Martinez	8954.02	
JD-16150	Justin Deggeller	8828.03	
JE-15715	Joe Elijah	8672.16	
LA-16780	Laura Armstrong	8631.52	
DR-12940	Daniel Raglin	8350.87	
NF-18385	Natalie Fritzler	8322.83	
KD-16270	Karen Daniels	8282.36	
NC-18535	Nick Crebassa	8241.74	
KD-16495	Keith Dawkins	8181.26	

Q5. Rank all customers based on total sales (Window Function)

SQL Query

```

1  WITH customer_sales AS (
2      SELECT
3          `Customer ID`,
4          `Customer Name`,
5          ROUND(SUM(Sales), 2) AS total_sales
6      FROM superstore_raw
7      GROUP BY
8          `Customer ID`,
9          `Customer Name`
10 )
11 SELECT
12     `Customer ID`,
13     `Customer Name`,
14     total_sales,
15     RANK() OVER (ORDER BY total_sales DESC) AS customer_rank
16 FROM customer_sales;

```

Output

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Customer ID	Customer Name	total_sales	customer_rank
SM-20320	Sean Miller	25043.05	1
TC-20980	Tamara Chand	19017.85	2
RB-19360	Raymond Buch	15117.34	3
TA-21385	Tom Ashbrook	14595.62	4
AB-10105	Adrian Barton	14355.61	5
SC-20095	Sanjit Chand	14142.33	6
KL-16645	Ken Lonsdale	14071.92	7
HL-15040	Hunter Lopez	12873.3	8
SE-20110	Sanjit Engle	12209.44	9
CC-12370	Christopher Conant	12129.07	10
TS-21370	Todd Sumrall	11885.87	11
GT-14710	Greg Tran	11820.12	12
BM-11140	Becky Martin	11609.9	13
SV-20365	Seth Vernon	11420.65	14

Result Grid Filter Rows: Export: Wrap Cell Content:				
	Customer ID	Customer Name	total_sales	customer_rank
	EH-13765	Edward Hooks	10005.98	20
	JL-15835	John Lee	9403	21
	GT-14635	Grant Thornton	9351.21	22
	HW-14935	Helen Wasserman	9300.25	23
	TB-21400	Tom Boeckenhauer	9012.74	24
	PF-19120	Peter Fuller	8980.5	25
	CM-12385	Christopher Martinez	8954.02	26
	JD-16150	Justin Deggeller	8828.03	27
	JE-15715	Joe Elijah	8672.16	28
	LA-16780	Laura Armstrong	8631.52	29
	DR-12940	Daniel Raglin	8350.87	30
	NF-18385	Natalie Fritzier	8322.83	31
	KD-16270	Karen Daniels	8282.36	32
	NC-18535	Nick Crebassa	8241.74	33

Q6. Assign row numbers to each order within a customer (Window Function + PARTITION BY)

SQL Query

Limit to 1000 rows				
1	•	SELECT		
2		`Customer ID`,		
3		`Customer Name`,		
4		`Order ID`,		
5		Sales,		
6		ROW_NUMBER() OVER (		
7		PARTITION BY `Customer ID`		
8		ORDER BY Sales DESC		
9		) AS row_num		
10		FROM superstore_raw;		

Output

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	Customer ID	Customer Name	Order ID	Sales	row_num
▶	AA-10315	Alex Avila	CA-2016-103982	3930.072	1
	AA-10315	Alex Avila	CA-2014-128055	673.568	2
	AA-10315	Alex Avila	CA-2016-103982	431.976	3
	AA-10315	Alex Avila	CA-2017-147039	362.94	4
	AA-10315	Alex Avila	CA-2014-128055	52.98	5
	AA-10315	Alex Avila	CA-2016-103982	41.72	6
	AA-10315	Alex Avila	CA-2015-121391	26.96	7
	AA-10315	Alex Avila	CA-2014-138100	14.94	8
	AA-10315	Alex Avila	CA-2014-138100	14.56	9
	AA-10315	Alex Avila	CA-2017-147039	11.54	10
	AA-10315	Alex Avila	CA-2016-103982	2.304	11
	AA-10375	Allen Arnold	CA-2016-131065	499.98	1
	AA-10375	Allen Arnold	CA-2015-140921	149.97	2
	AA-10375	Allen Arnold	CA-2017-100230	116.98	3
	AA-10375	Allen Arnold	CA-2015-114503	84.96	4
	AA-10375	Allen Arnold	US-2017-169488	39.96	5

Result Grid					
Filter Rows:					
Export: Wrap Cell Content:					
	Customer ID	Customer Name	Order ID	Sales	row_num
	AA-10645	Anna Andreadi	CA-2014-154963	853.93	3
	AA-10645	Anna Andreadi	CA-2014-143210	271.9	4
	AA-10645	Anna Andreadi	CA-2014-154963	170.058	5
	AA-10645	Anna Andreadi	CA-2016-131205	129.92	6
	AA-10645	Anna Andreadi	CA-2016-131205	113.94	7
	AA-10645	Anna Andreadi	CA-2015-110863	106.32	8
	AA-10645	Anna Andreadi	CA-2014-154963	82.782	9
	AA-10645	Anna Andreadi	CA-2016-131205	42.6	10
	AA-10645	Anna Andreadi	CA-2016-147137	27.92	11
	AA-10645	Anna Andreadi	CA-2016-147137	25.4	12
	AA-10645	Anna Andreadi	CA-2017-157196	12.96	13
	AA-10645	Anna Andreadi	CA-2014-143210	9.82	14
	AA-10645	Anna Andreadi	CA-2016-131205	5.28	15
	AB-10015	Aaron Bergman	CA-2016-140935	341.96	1
	AB-10015	Aaron Bergman	CA-2014-156587	242.94	2
	AB-10015	Aaron Bergman	CA-2016-140935	221.98	3

Q7. Display Top 3 Customers based on Total Sales (Window Function)

SQL Query

```

1  WITH customer_sales AS (
2      SELECT
3          `Customer ID`,
4          `Customer Name`,
5          ROUND(SUM(Sales), 2) AS total_sales
6      FROM superstore_raw
7      GROUP BY
8          `Customer ID`,
9          `Customer Name`
10 ),
11 ranked_customers AS (
12     SELECT
13         *,
14         RANK() OVER (ORDER BY total_sales DESC) AS customer_rank
15     FROM customer_sales
16 )
17 SELECT *
18 FROM ranked_customers
19 WHERE customer_rank <= 3;

```

Output

Result Grid				
Filter Rows:				
Export: Wrap Cell Content:				
	Customer ID	Customer Name	total_sales	customer_rank
▶	SM-20320	Sean Miller	25043.05	1
	TC-20980	Tamara Chand	19017.85	2
	RB-19360	Raymond Buch	15117.34	3

4. Final Combined Query

Display Customer Name, Total Sales and Rank using JOIN + CTE + Window Function

SQL Query

```
1 WITH customer_sales AS (  
2     SELECT  
3         `Customer ID`,  
4         ROUND(SUM(Sales), 2) AS total_sales  
5     FROM superstore_raw  
6     GROUP BY `Customer ID`  
7 )  
8 SELECT  
9     c.customer_id,  
10    c.customer_name,  
11    cs.total_sales,  
12    RANK() OVER (  
13        ORDER BY cs.total_sales DESC  
14    ) AS customer_rank  
15 FROM customer_sales cs  
16 JOIN customers c  
17 ON cs.`Customer ID` = c.customer_id  
18 ORDER BY customer_rank;
```

Output

Result Grid				
Filter Rows:		Export: 		
Wrap Cell Content: 				
	customer_id	customer_name	total_sales	customer_rank
▶	SM-20320	Sean Miller	25043.05	1
	TC-20980	Tamara Chand	19017.85	2
	RB-19360	Raymond Buch	15117.34	3
	TA-21385	Tom Ashbrook	14595.62	4
	AB-10105	Adrian Barton	14355.61	5
	SC-20095	Sanjit Chand	14142.33	6
	KL-16645	Ken Lonsdale	14071.92	7
	HL-15040	Hunter Lopez	12873.3	8
	SE-20110	Sanjit Engle	12209.44	9
	CC-12370	Christopher Conant	12129.07	10
	TS-21370	Todd Sumrall	11885.87	11
	GT-14710	Greg Tran	11820.12	12
	BM-11140	Becky Martin	11609.9	13
	SV-20365	Seth Vernon	11420.65	14
	CJ-12010	Caroline Jumper	11079.74	15
	CL-12565	Clay Ludtke	10880.55	16

Result Grid				
Filter Rows:		Export:	Wrap Cell Content:	
customer_id	customer_name	total_sales	customer_rank	
GK-14620	Grace Kelly	6497.27	60	
JA-15970	Joseph Airdo	6475.47	61	
NM-18445	Nathan Mautz	6459.34	62	
JD-15895	Jonathan Doherty	6456.46	63	
SB-20185	Sarah Brown	6411	64	
JE-15745	Joel Eaton	6400.45	65	
JG-15160	James Galang	6366.39	66	
CM-12655	Corinna Mitchell	6339.56	67	
VD-21670	Valerie Dominguez	6283.35	68	
MJ-17740	Max Jones	6282.23	69	
BH-11710	Brosina Hoffman	6255.35	70	
DM-13015	Darrin Martin	6247.37	71	
DR-12880	Dan Reichenbach	6247.21	72	
RL-19615	Rob Lucas	6234.91	73	
VW-21775	Victoria Wilson	6112.7	74	
QJ-19255	Quincy Jones	6108.34	75	

5. Mini Project – Customer Sales Insights

5.1 Top 5 Customers

SQL Query

Limit to 1000 rows				
1	•	SELECT		
2		`Customer ID`,		
3		`Customer Name`,		
4		ROUND(SUM(Sales),2) AS total_sales		
5		FROM superstore_raw		
6		GROUP BY		
7		`Customer ID`,		
8		`Customer Name`		
9		ORDER BY total_sales DESC		
10		LIMIT 5;		

Output

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	Customer ID	Customer Name	total_sales
▶	SM-20320	Sean Miller	25043.05
	TC-20980	Tamara Chand	19017.85
	RB-19360	Raymond Buch	15117.34
	TA-21385	Tom Ashbrook	14595.62
	AB-10105	Adrian Barton	14355.61

5.2 Bottom 5 Customers

SQL Query

```
1 • SELECT
2     `Customer ID`,
3     `Customer Name`,
4     ROUND(SUM(Sales),2) AS total_sales
5 FROM superstore_raw
6 GROUP BY
7     `Customer ID`,
8     `Customer Name`
9 ORDER BY total_sales ASC
10 LIMIT 5;
```

Output

Customer ID	Customer Name	total_sales
TS-21085	Thais Sissman	4.83
LD-16855	Lela Donovan	5.3
MG-18205	Mitch Gastineau	12.32
CJ-11875	Carl Jackson	16.52
RS-19870	Roy Skaria	22.33

5.3 Customers with Only One Order

SQL Query

```
1 • SELECT
2     `Customer ID`,
3     `Customer Name`,
4     COUNT(DISTINCT `Order ID`) AS total_orders
5 FROM superstore_raw
6 GROUP BY
7     `Customer ID`,
8     `Customer Name`
9 HAVING COUNT(DISTINCT `Order ID`) = 1;
```

Output

Customer ID	Customer Name	total_orders
AO-10810	Anthony O'Donnell	1
AR-10570	Anemone Ratner	1
CJ-11875	Carl Jackson	1
JC-15385	Jenna Caffey	1
JR-15700	Jocasta Rupert	1
LD-16855	Lela Donovan	1
MG-18205	Mitch Gastineau	1
PH-18790	Patricia Hirasaki	1
RE-19405	Ricardo Emerson	1
RM-19750	Roland Murray	1
SM-20905	Susan MacKendrick	1
TC-21145	Theresa Coyne	1

5.4 Customers with Above Average Sales

SQL Query

```
1 WITH customer_sales AS (  
2     SELECT  
3         `Customer ID`,  
4         `Customer Name`,  
5         ROUND(SUM(Sales),2) AS total_sales  
6     FROM superstore_raw  
7     GROUP BY  
8         `Customer ID`,  
9         `Customer Name`  
10 )  
11 SELECT *  
12 FROM customer_sales  
13 WHERE total_sales >  
14 (  
15     SELECT AVG(total_sales)  
16     FROM customer_sales  
17 )  
18 ORDER BY total_sales DESC;
```

Output

Result Grid			
Filter Rows:		Export:	Wrap Cell Content: IA
	Customer ID	Customer Name	total_sales
▶	SM-20320	Sean Miller	25043.05
	TC-20980	Tamara Chand	19017.85
	RB-19360	Raymond Buch	15117.34
	TA-21385	Tom Ashbrook	14595.62
	AB-10105	Adrian Barton	14355.61
	SC-20095	Sanjit Chand	14142.33
	KL-16645	Ken Lonsdale	14071.92
	HL-15040	Hunter Lopez	12873.3
	SE-20110	Sanjit Engle	12209.44
	CC-12370	Christopher Conant	12129.07
	TS-21370	Todd Sumrall	11885.87
	GT-14710	Greg Tran	11820.12
	BM-11140	Becky Martin	11609.9
	SV-20365	Seth Vernon	11420.65

	Customer ID	Customer Name	total_sales
	ZC-21910	Zuschuss Carroll	8021.14
	JH-15985	Joseph Holt	7955
	NP-18700	Nora Preis	7903.18
	AH-10690	Anna Häberlin	7888.29
	JE-15610	Jim Epp	7754.98
	AB-10060	Adam Bellavance	7732.67
	JW-15220	Jane Waco	7721.71
	JM-15865	John Murray	7625.08
	RW-19540	Rick Wilson	7397.4
	LC-16885	Lena Creighton	7396.77
	PO-18850	Patrick O'Brill	7367.33
	BM-11650	Brian Moss	7283.82
	NW-18400	Natalie Webber	7234.01
	DN-13740	Dean nercer	7198.76

5.5 Highest Order Value for Each Customer

SQL Query

```

1 • SELECT
2     s.`Customer ID`,
3     s.`Customer Name`,
4     s.`Order ID`,
5     s.Sales
6 FROM superstore_raw s
7 JOIN (
8     SELECT
9         `Customer ID`,
10        MAX(Sales) AS max_sales
11    FROM superstore_raw
12   GROUP BY `Customer ID`
13 ) m
14 ON s.`Customer ID` = m.`Customer ID`
15 AND s.Sales = m.max_sales
16 ORDER BY s.Sales DESC;

```

Output

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Customer ID	Customer Name	Order ID	Sales
SM-20320	Sean Miller	CA-2014-145317	22638.48
TC-20980	Tamara Chand	CA-2016-118689	17499.95
RB-19360	Raymond Buch	CA-2017-140151	13999.96
TA-21385	Tom Ashbrook	CA-2017-127180	11199.968
HL-15040	Hunter Lopez	CA-2017-166709	10499.97
AB-10105	Adrian Barton	CA-2016-117121	9892.74
SC-20095	Sanjit Chand	CA-2014-116904	9449.95
BS-11365	Bill Shonely	US-2016-107440	9099.93
SE-20110	Sanjit Engle	CA-2016-158841	8749.95
CC-12370	Christopher Conant	CA-2016-143714	8399.976
KL-16645	Ken Lonsdale	CA-2014-143917	8187.65
BM-11140	Becky Martin	CA-2014-139892	8159.952
GT-14635	Grant Thornton	US-2017-168116	7999.98

Result Grid   Filter Rows: Export:  Wrap Cell Content: 				
	Customer ID	Customer Name	Order ID	Sales
	SA-20830	Sue Ann Reed	CA-2014-154627	2735.952
	MW-18235	Mitch Willingham	CA-2014-117639	2715.93
	DM-13345	Denise Monton	CA-2017-157854	2690.97
	AS-10285	Alejandro Savely	US-2016-164196	2678.94
	LC-16960	Lindsay Castell	CA-2015-111038	2676.672
	SF-20200	Sarah Foster	CA-2017-123967	2665.62
	AT-10735	Annie Thurman	CA-2015-123113	2625.12
	JL-15835	John Lee	CA-2014-127299	2624.985
	ED-13885	Emily Ducich	CA-2015-160227	2621.322
	GK-14620	Grace Kelly	US-2016-116729	2575.944
	BD-11320	Bill Donatelli	CA-2015-120677	2567.84
	JE-15715	Joe Elijah	CA-2016-169670	2563.056
	SR-20425	Sharelle Roach	CA-2017-134845	2549.985

Conclusion

This assignment demonstrates the use of advanced SQL concepts, including Subqueries, Common Table Expressions (CTEs), Window Functions, JOIN operations, and business-oriented analytical queries using the Superstore dataset. The dataset was normalized into separate tables and analyzed to generate meaningful business insights.